Parallel Bigram and Trigram Analysis: Performance Optimization with OpenMP

Lorenzo Cappetti

January 2026

UNIVERSITÀ
DEGLI STUDI
FIRENZE

UNIVERSITÀ DEGLI STUDI FIRENZE

Table of Contents

Problem statement & pipeline

- **Dataset**: $\sim$**1500** Project Gutenberg books, over **600 MB** of text.
- **Goal**: extract bigrams/trigrams (word-level and char-level) for linguistic analysis.
- **Pipeline**: load $\rightarrow$ normalization $\rightarrow$ tokenization $\rightarrow$ n-gram counting $\rightarrow$ output.
- **Constraint**: large corpus $\Rightarrow$ optimize end-to-end.
- **Idea**: OpenMP parallelization on multi-core CPUs.

Key point

A *data-intensive* problem: scalability depends not only on compute, but also on **I/O and memory**.

Parallelization strategy: book-level work units

- Work unit: **one book** $\rightarrow$ processed independently.
- **Embarrassingly parallel** workload: no dependencies across books.
- Each thread executes the **entire pipeline**.
- Synchronization is limited to the **global update** of frequency counts.

Why it works

Many work units ($\sim$1500 books) + high size variability $\Rightarrow$ good load balancing is required.

Table of Contents

2 Approach

UNIVERSITÀ DEGLI STUDI FIRENZE

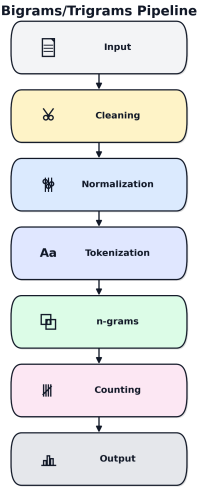
Project approach: three implementations

- **Sequential (baseline).**
  - Single-thread end-to-end execution: *load* → *clean* → *normalize* → *tokenize* → *n-gram*.
  - Used as a reference for: total runtime, and result correctness.
- **Parallel (OpenMP, AoS).**
  - Work is split by **book/file** across threads.
  - N-gram counting with **sharded maps** to reduce contention (final merge).
  - Goal: exploit parallelism while keeping a “traditional” data structure (Array of Structs).
- **Parallel (OpenMP, SoA).**
  - Same parallelization strategy, but using a **Structure of Arrays** layout.
  - Better **cache locality** and more linear accesses; less overhead from allocations/intermediate objects.

Sequential Implementation (Baseline)

2 Approach

- **Single-thread baseline** (end-to-end).
- **Per-book workflow**:
  - Load (binary).
  - Gutenberg cleaning (START/END).
  - Normalization (lowercase, punctuation, UTF-8 accents).
  - Tokenization:
    - Word tokens.
    - Character stream.
  - N-gram counting:
    - Word bigrams / trigrams.
    - Character bigrams / trigrams.
  - Aggregate + sort + output (CSV + stats).



Parallelization (OpenMP): book-level tasks

2 Approach

- **Work unit**: one book $\rightarrow$ full pipeline on a single thread.
- **OpenMP structure**:
  - `#pragma omp parallel` + `#pragma omp for schedule(dynamic)`.
  - **Dynamic scheduling** to balance books with highly different sizes.
  - **Per-thread buffers**: word tokens + character stream (no sharing).
- **Global counting (controlled concurrency)**:
  - Four global structures: word/char $\times$ bi/tri-grams.
  - **Sharded maps** (many shards) $\rightarrow$ fine-grained locking.
  - Merge/sort **outside** the parallel region.

UNIVERSITÀ DEGLI STUDI FIRENZE

Optimizations implemented

2 Approach

- **I/O & preprocessing**:
  - Binary read + pre-sized buffer.
  - Cleaning + normalization **in-place** (START/END markers).
- **Tokenization & memory**:
  - Word tokens via `string_view` (zero-copy).
  - Buffer **reserve** to reduce reallocations.
- **Counting structures**:
  - **Sharding** to reduce contention.
  - **Arena/pool** for string storage (fewer `malloc`/free calls).
  - Cache alignment to mitigate *false sharing*.
- **Data layout**:
  - **SoA** variant (separating arrays/metadata) for better locality and lower overhead.

Parallel Implementations: AoS vs SoA

2 Approach

OpenMP + AoS (parallel.cpp).

- `Shard{mutex,map,arena}` $\times$ `NUM_SHARDS`.
- Sharding: `idx` $=$ `hash(key)` $\bmod$ `NUM_SHARDS`.
- Update: lock $\rightarrow$ `find/insert++` $\rightarrow$ optional copy into `Arena`.
- **Pros**: compact “per-shard” access, simple/extensible code.
- **Cons**: less suitable for “per-field” optimizations (false sharing mitigated via `alignas(64)`).

OpenMP + SoA (parallel_soa.cpp).

- `mutexes[i]`, `maps[i]`, `arenas[i]`.
- SoA also inside `ArenaSoA`: `data[k]`, `size[k]`, `used[k]`.
- Same OpenMP: `schedule(dynamic)` and identical sharding.
- **Pros**: data/metadata separation, potential gains for “per-field” scans.
- **Cons**: more bookkeeping; cache benefits are not guaranteed for per-key updates.

UNIVERSITÀ DEGLI STUDI FIRENZE

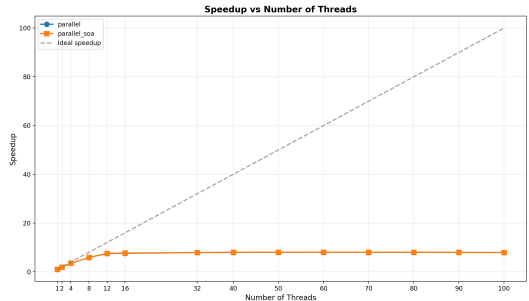
Table of Contents

3 Results

Results: Speedup vs #Threads

3 Results

- **Metric**: $S(p) = T_{seq}/T_p$.
- **Strong scaling** up to **8–12 threads**.
- At **12 threads**: $\sim$ **6.3–6.5**$\times$ speedup.
- Beyond **12–16**: **flat** curve (minimal gains).
- Even at **50–100 threads**: $\sim$ **6.8**$\times$.
- **Bottleneck**: file loading + memory bandwidth saturation.
- **SoA** slightly better than AoS (small but consistent advantage).

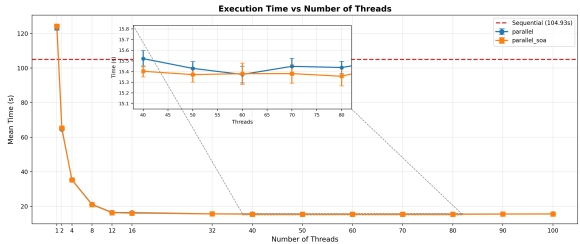


Speedup vs #Threads (AoS vs SoA + ideal).

Results: Execution Time vs #Threads

3 Results

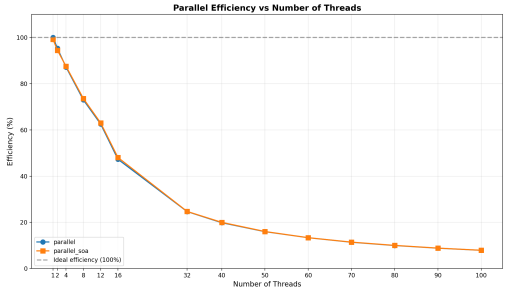
- **Baseline**: sequential $\approx$ 104.9s.
- Sharp reduction up to **8–12 threads**.
- **Plateau** beyond $\sim$12–16 threads: $T_p \approx 15.3$–15.6s.
- **Best speedup** $\approx 104.9/15.4 \simeq 6.8\times$ ($\approx$ 60 threads).
- **SoA vs AoS**: SoA slightly better (small but stable margin).
- **Diminishing returns**: beyond 40+ threads, minimal gains (I/O/memory saturation).



Execution time vs number of threads.

Results: Parallel Efficiency vs #Threads

- **Efficiency**: $E(p) = \frac{S(p)}{p}$.
- High for a few threads, then it drops quickly:
  - $\sim$90% @ 4 threads, $\sim$70% @ 8 threads.
  - $\sim$50% @ 16 threads.
  - $< 20\%$ beyond 40 threads.
- **Interpretation**: overhead + saturation (I/O / memory) $\Rightarrow$ diminishing returns.
- **SoA** $\approx$ AoS: small differences, with SoA slightly better.



Parallel efficiency vs threads: AoS vs SoA, dashed line = ideal (100%).

UNIVERSITÀ DEGLI STUDI FIRENZE

Table of Contents

4 Conclusions

Conclusions

4 Conclusions

- The problem is **data-intensive**: performance is limited by **I/O** and **memory bandwidth**, not only by compute.
- With OpenMP (book-level) we achieve good **strong scaling** up to $\sim$ **12–16 threads**.
- Observed **maximum speedup**: $\sim$ **6.8**$\times$ (plateau $\approx$ 15.4s vs 104.9s).
- Beyond 16 threads: **diminishing returns** (efficiency $<$ 20% at high thread counts).
- The **SoA** variant is **slightly better** than AoS: small but consistent benefits.
- Concurrent data structures (**sharding** + fine-grained locks) make global updates scalable and stable.

Takeaway

Parallelization works, but the final limit is set by the **end-to-end pipeline** (I/O + memory).

Lessons Learned

4 Conclusions

- **End-to-end measurement**: optimizing only the kernel is not enough if I/O and preprocessing dominate.
- **Load balancing matters**: heterogeneous dataset $\Rightarrow$ `schedule(dynamic)` is often better than static.
- **Contention**: a single global map does not scale $\Rightarrow$ **sharding** is a practical solution.
- **Data layout**: SoA improves locality and reduces overhead, but increases code complexity.
- **Memory and allocations**: preallocating buffers / reusing strings greatly reduces performance noise.
- **Reproducibility**: running multiple trials and reporting variability (CV%) helps validate results.

If I had more time

Thread pinning/affinity, async I/O, and a more efficient merge to further reduce the plateau.

UNIVERSITÀ DEGLI STUDI FIRENZE

Parallel Bigram and Trigram Analysis: Performance Optimization with OpenMP

Thank you for listening!

UNIVERSITÀ DEGLI STUDI FIRENZE

References

4 Conclusions

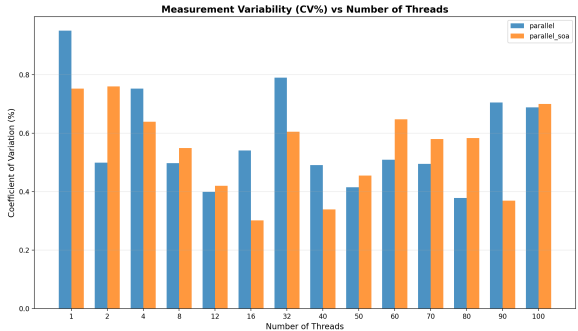
OpenMP Architecture Review Board,
OpenMP API Specification.
openmp.org/specifications

Project Gutenberg,
Project Gutenberg Digital Library.
gutenberg.org

GitHub,
Repository link for code and report.
Cappetti99/Bigrams_Trigrams

Backup: Measurement Variability (CV%) vs #Threads

- **CV%** = $\sigma/\mu \cdot 100$ across runs (measurement stability).
- **Low** variability over the whole range: $\approx$ 0.3–0.9%.
- No clear trend with thread count $\Rightarrow$ **robust** results.
- SoA and AoS: minimal differences, same order of magnitude.



Coefficient of Variation (CV%) across runs vs number of threads (AoS vs SoA).